

Day-1 Hand Tracking Code Explanation

Date: 15/01/2026

Below is a detailed line-by-line explanation of the Day 1 hand tracking code using MediaPipe and OpenCV.

Code Line / Section:

```
import cv2
```

Explanation: Imports the OpenCV library to handle video capture, image processing, and display. cv2 is required to access webcam frames and manipulate images.

```
import mediapipe as mp
```

Explanation: Imports the MediaPipe library, which provides the hand landmark detection model and utilities for tracking hands in real time.

```
mp_hands = mp.solutions.hands
```

Explanation: Accesses the hands module from MediaPipe, which contains the pre-trained hand landmark detection model and associated methods.

```
cap = cv2.VideoCapture(0)
```

Explanation: Initializes video capture from the default webcam (device 0). This allows us to read live video frames for processing.

```
with mp_hands.Hands(min_detection_confidence=0.7, min_tracking_confidence=0.5) as hands:
```

Explanation: Creates a MediaPipe Hands object with specified confidence thresholds. min_detection_confidence ensures the hand is detected reliably, min_tracking_confidence ensures landmarks are consistently tracked across frames.

```
while cap.isOpened(): ret, frame = cap.read()
```

Explanation: Starts a loop that runs while the webcam is active. cap.read() reads the current video frame; ret confirms if the frame is successfully captured, frame contains the image.

```
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

Explanation: Converts the frame from BGR (OpenCV default) to RGB because MediaPipe expects images in RGB format for processing.

```
results = hands.process(frame)
```

Explanation: Feeds the current frame to the MediaPipe Hands model to detect hands and extract landmark coordinates.

```
frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
```

Explanation: Converts the processed frame back to BGR so OpenCV can display it correctly.

```
if results.multi_hand_landmarks: for hand_landmarks in results.multi_hand_landmarks:  
    mp.solutions.drawing_utils.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
```

Explanation: Checks if any hands are detected. If yes, loops through each hand and draws the landmarks and connections on the frame for visualization.

```
cv2.imshow('Hand Tracking', frame)
```

Explanation: Displays the frame with hand landmarks in a window named 'Hand Tracking'.

```
if cv2.waitKey(1) & 0xFF == ord('q'): break
```

Explanation: Waits for a key press. If 'q' is pressed, the loop breaks and the program exits cleanly.

```
cap.release() cv2.destroyAllWindows()
```

Explanation: Releases the webcam resource and closes all OpenCV windows, cleaning up the program before exit.

This explanation allows Day 1's code to be easily understood, updated, and used for future reference or further development.